INTERVIEW BRIEFING · PROJECT EXPLANATION

My Codebase Onboarding Assistant

A source-grounded assistant that helps developers understand an unfamiliar repository by indexing its files, retrieving relevant code, and answering questions with citations.

THE 20-SECOND VERSION

I built a codebase Q&A and onboarding tool. FastAPI accepts projects and questions, Celery/Redis run indexing in the background, and Chroma powers repository-scoped RAG. A bounded tool-using agent can inspect files when retrieval is not enough, while guardrails check scope, paths, and model-bound tool output. The result is a cited answer grounded in the selected codebase.

What it is designed to solve

Joining an unfamiliar codebase often means manually hunting through entry points, routes, configuration, dependencies, and execution paths. This project gives an engineer one place to ask those questions in plain language and receive an answer linked back to repository files.

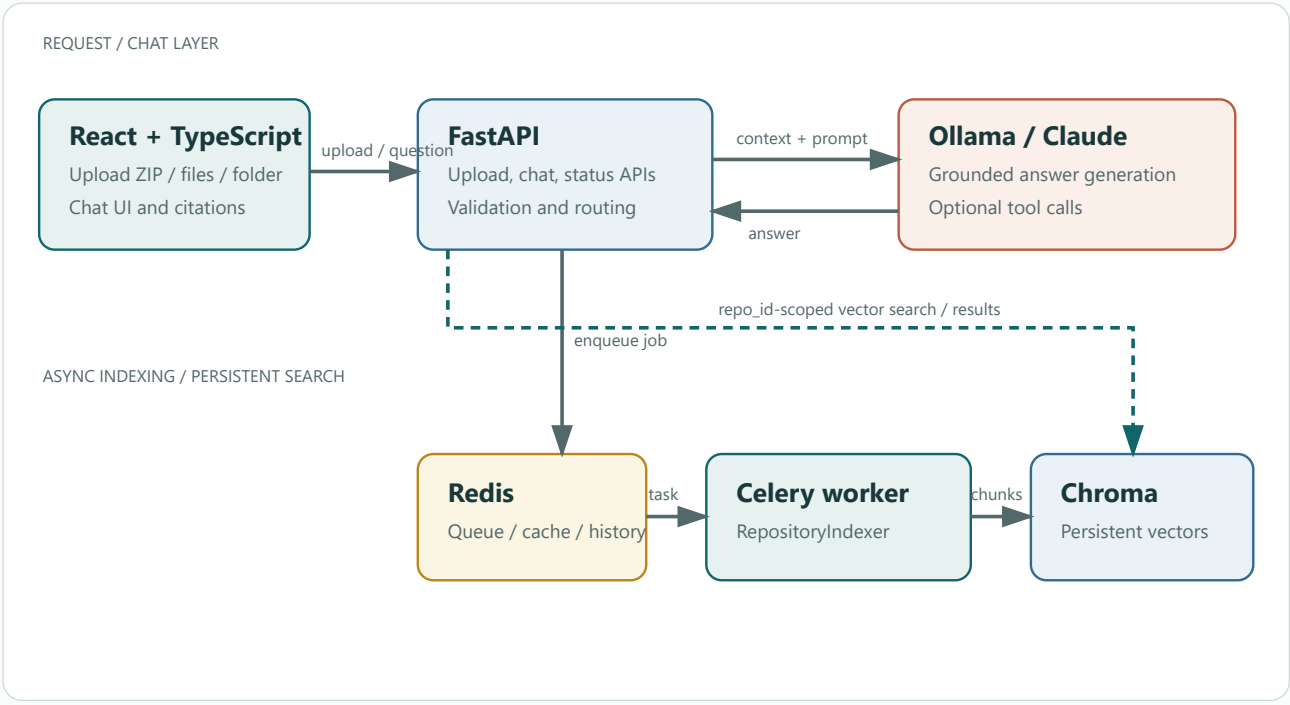
Repository onboarding | RAG search | Async indexing | Cited answers | Multi-repository | Security guardrails

Prepared as an interview speaking guide · Codebase verified: my-codebase-onboarding-assistant-2

01 · SYSTEM DESIGN

How the application works

The system separates the interactive API from slower indexing work. The API accepts uploads and questions; Celery processes repositories in the background; Chroma stores searchable chunks; the agent uses retrieved context and repository tools to form an answer.



Solid arrows show upload/index and model request paths. The dashed arrow represents repository-scoped retrieval for a question. Redis also backs chat history and response caching; Docker Compose runs the services together.

1. **Receive:** the frontend sends a ZIP or raw file; folders and multiple files are packaged into a ZIP in the browser.
2. **Queue:** FastAPI validates the upload, stores it on a volume shared with Celery, registers a repository ID, and queues indexing.
3. **Index:** Celery scans eligible files, chunks text, attaches file/line/repository metadata, and writes chunks to Chroma.
4. **Answer:** questions retrieve relevant chunks for that repository, then the agent generates a response with file citations.

02 · TECHNOLOGY CHOICES

Major technology stack

Each technology has a clear job: web interaction, API orchestration, background processing, semantic retrieval, or answer generation.

Layer	Technology	Role in this project
Frontend	React 18, TypeScript, Vite	Repository picker, ZIP/file/folder intake, chat interface, progress and citations. Axios calls the API; `fflate` packages folder selections into ZIP.
Backend API	Python, FastAPI, Uvicorn	Exposes chat, upload, repository, job-status, health, and webhook endpoints. Pydantic-based settings and request models support configuration and validation.
Background jobs	Celery + Redis	Long indexing work runs outside the request/response path. Redis brokers tasks and also supports chat history and answer caching.
Vector retrieval	ChromaDB	Persistent vector collection stores chunks with repository ID, file path, line range, and other metadata. Search filters by repository ID.
Answer models	Ollama / Claude	Provider selection is configurable. Ollama is the default and uses the configured local model (default `gemma4:26b`); Claude is an API-backed option.
Code intelligence	GitPython, Git CLI	Reads commit metadata and supports incremental indexing for Git updates; webhook handling can queue repository refreshes.
Deployment	Docker Compose	Runs frontend, API, Redis, Celery worker, and Celery Beat with shared repository, upload, and vector-store volumes.
Quality	Pytest	Unit and integration tests cover indexing, repository tools, uploads, security, and webhook behavior.

Accurate embedding explanation: the codebase contains an `EmbeddingProvider` helper using Sentence Transformers and `all-MiniLM-L6-v2`. The active `RepositoryIndexer` calls Chroma's collection API directly and does not call that helper; describe Sentence Transformers as an included helper/dependency, not as a separately invoked stage in the active indexing loop.

API process: `backend/app/main.py`

Creates the FastAPI object, configures CORS/logging, mounts the main API router and webhook router, and defines health/readiness endpoints plus startup/shutdown hooks. Uvicorn launches it as `app.main:app`.

UI composition: `frontend/src/App.tsx`

Loads repositories, manages the selected project and upload state, polls indexing status, triggers the overview question, and composes the repository picker, intake panel, chat, status, and job trace components.

About "app.py": there is no backend `app.py` in the active project. If an interviewer asks what `app.py` does, clarify that this project uses `backend/app/main.py` for backend startup and `frontend/src/App.tsx` for the frontend's top-level UI.

03 · CORE ARCHITECTURE

How the five core technologies fit

These are connected responsibilities, not isolated buzzwords: FastAPI coordinates requests, Celery handles long-running indexing, RAG supplies relevant project evidence, the agent chooses bounded tools, and guardrails constrain unsafe inputs and outputs.

FastAPI · API and orchestration

`backend/app/main.py` starts the service. Routers accept uploads and chat, validate repository IDs, queue work, expose job status, and stream answers to the React client.

RAG · evidence retrieval

The indexer chunks eligible text and stores it with file, line, and `repo_id` metadata in Chroma. Each question retrieves the closest chunks from only the selected repository; the answer is generated from that context.

Agentic AI · bounded tool use

One agent coordinates the LLM and tools: repository grep, line-range file reads, and Git blame. When context is insufficient, the provider may request a tool; execution is bounded to at most four iterations. This is tool-using agentic AI, not a multi-agent team.

Celery + Redis · background work

FastAPI enqueues indexing instead of blocking on it. A Celery worker scans and indexes the uploaded project; Redis carries jobs and also supports job state, chat history, and response caching.

Guardrails · safer boundaries

Scope checks keep chat code-focused; upload/path checks reject unsafe entries; file discovery excludes secrets and binaries; tool output is scanned and redacted before it is sent to the LLM. Responses are checked before returning to the user.

Chroma + Ollama / Claude

Chroma persists searchable vectors. The provider setting selects local Ollama (default) or Claude. Provider prompts require repository evidence and exact file paths instead of invented claims.

One complete request path

FastAPI receives project upload	Celery indexes repository chunks	Chroma retrieves matching evidence	Agent uses tools only if needed
Guardrails scan tool evidence	LLM writes a cited answer	FastAPI streams the response	React displays answer + sources

How I would describe the design choice

“I separated the system into a request layer and a background indexing layer. RAG keeps answers tied to the right repository, and the agent can inspect source files when retrieval alone is not enough. I bounded those tool calls and added guardrails around uploaded paths and model context, so the system is useful without treating repository content as trusted instructions.”

Accuracy note: this project has one tool-using agent loop with a maximum of four tool iterations. It should be presented as an agentic workflow, not as multiple collaborating AI agents.

04 · REQUEST LIFECYCLE

From upload to an answer

Indexing and question answering are separate paths. This keeps large repositories from blocking the browser request and gives every vector a repository identity.

A. Project intake and indexing

- User chooses one file, several files, a folder, or a ZIP.
- The browser converts folders/files to a ZIP while retaining relative paths.
- FastAPI validates paths, size, and archive members; it ignores unsafe entries and unsupported archive formats.
- The archive is stored under the shared upload volume and registered with a unique `repo_id`.
- Celery runs `RepositoryIndexer` asynchronously and reports job state through Redis.
- Eligible text files are chunked; Chroma stores chunks and source metadata for later retrieval.

B. Question and answer

- The frontend sends the question with the selected `repo_id`.
- The agent searches Chroma for relevant chunks, scoped to that repository.
- If needed, the model can call repository-root-scoped grep/read/Git tools.
- Tool output is scanned for secrets and insecure patterns before reaching the model.
- The configured Ollama or Claude provider writes an evidence-based answer.
- The API streams the response, citations, and tool trace back to the UI.

Why this design is useful

Grounded answers

Answers use retrieved repository chunks and can cite their source paths and line ranges.

Responsive intake

Celery handles indexing in the background; the UI can poll status while the API remains available.

Repository isolation

Vectors and direct file tools are scoped to the selected repo, preventing one project's context from being used for another.

Trust and safety

Repository scanning excludes common generated folders, binary files, and sensitive files. ZIP extraction rejects traversal and symlink entries. Direct tools constrain paths to a repository root. The LLM tool pipeline sanitizes tool output and redacts detected secrets before adding it to model context.

> **Useful distinction:** semantic retrieval supplies the first context; it is not the whole repository pasted into the prompt. The current agent retrieves five top-ranked chunks initially and may use up to four tool iterations for supported providers when it needs more evidence.

05 · INTERVIEW PRACTICE

How to explain what you built

One-minute answer

"I built a codebase onboarding assistant to reduce the time developers spend figuring out an unfamiliar project. Users can upload a code file or project archive/folder, and the backend indexes its readable files asynchronously. I store chunks with file and repository metadata in Chroma, retrieve relevant context for each question, and use an LLM to produce a concise answer with citations. FastAPI handles the API, Celery and Redis keep indexing off the request path, and the React/TypeScript frontend provides the upload and chat experience. I also added repository isolation and security checks so tool reads stay inside the selected project and sensitive output is filtered before it reaches the model."

"How did you build it?"

1. **Define the workflow:** select a project, index it, ask a question, and see source-backed answers.
2. **Build the service boundary:** use FastAPI for repository, upload, chat, and job-status endpoints.
3. **Make indexing asynchronous:** put Celery and Redis between uploads and expensive indexing work.
4. **Add retrieval:** discover files, filter unsafe/unhelpful content, chunk text, and store repository-scoped vectors.
5. **Ground the assistant:** retrieve evidence, allow bounded file tools, add citations, and support configurable LLM providers.
6. **Harden and test:** add path/archive validation, secret scanning, cache/history isolation, and focused tests.

Present this as the architecture and engineering approach you implemented; adjust the order if your actual development timeline differed.

Major advantage

It combines repository-specific retrieval, direct code tools, citations, asynchronous indexing, and multi-repository isolation in one onboarding workflow. The model is guided by the selected source rather than being asked to guess from a generic prompt.

Trade-offs to acknowledge

Retrieval quality depends on chunking and the top-five initial search; ambiguous questions may need follow-up. RAR/7z archives must be extracted or converted. The evaluation runner is currently a scaffold, so a useful next step is a measured benchmark with representative questions and citation/answer-quality checks.

Questions you can answer confidently

Why use RAG instead of sending the whole codebase to an LLM?

"Repositories can be large, and sending everything is expensive and noisy. I index chunks once, retrieve the most relevant evidence per question, and keep source metadata for citations."

Why Celery and Redis?

"Indexing can take longer than a normal API request. Celery runs it in the background; Redis transports tasks and lets the UI poll progress."

What would you improve next?

"I would add a benchmark dataset, measure retrieval recall and citation accuracy, tune chunking/top-k from those results, and expose indexing coverage so users know which files were included."

Code landmarks: `backend/app/main.py` · `backend/app/api/routes.py` · `backend/app/tasks/reindex.py` · `backend/app/indexer/indexer.py` · `backend/app/services/vector_store.py` · `backend/app/agent/loop.py` · `backend/app/services/llm_service.py` · `frontend/src/App.tsx`
